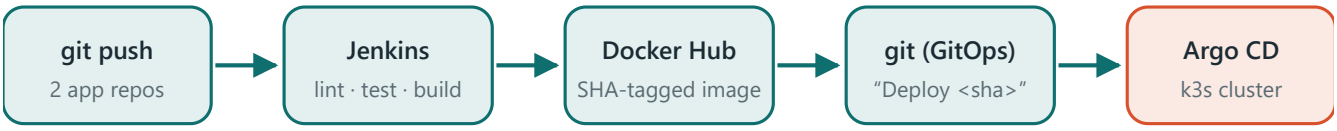
DEVOPS COURSE · FINAL PROJECT

TripSync

An offline-first, map-first trip planner, hosted for real on a homelab Kubernetes cluster and delivered by a fully automated pipeline.



Author: Roye · **Course:** Technion DevOps · **Date:** November 2026

Live: https://tripsync.royerandomnonsense.uk · **Code:** github.com/Swagarot (tripsync-backend, tripsync-frontend, tripsync-devops)

Contents

1 Overview and goals

2 Architecture

3 The application

4 Continuous integration: the Jenkins pipelines

5 Continuous delivery: GitOps with Argo CD

6 Kubernetes and the Helm chart

7 Infrastructure as code: Terraform for k3s and AWS EKS

8 Storage, data and backups

9 Networking and exposure

10 Observability: metrics, logs and alerts

11 Security

12 Testing

13 Incidents and lessons learned

14 Project timeline

15 Limitations and future work

A Appendix: versions and repository layout

1. Overview and goals

TripSync is a multi-user trip planner: you build a trip day by day on a map, share it with friends, split the costs, and keep using it with no connection. It is installable as an app (PWA) and is publicly hosted at **tripsync.royerandomnonsense.uk**.

The app is the vehicle. The point of the project is everything around it: how the code gets from a commit to a running, monitored, backed-up service without anyone touching a server. Every part of that path is automated and kept in git:

- **Three repositories**, as the teachers asked: `tripsync-backend` and `tripsync-frontend`, each with its own **Jenkinsfile** (two independent pipelines), and `tripsync-devops` with the Helm chart, Terraform, monitoring configuration and the project's `CLAUDE.md`.
- **CI** in Jenkins: lint, unit tests against a real throwaway database, image build tagged with the commit SHA, push to Docker Hub.
- **CD with GitOps**: Jenkins records the new image tag in git and **Argo CD**, running inside the cluster, makes the cluster match git.
- **Infrastructure as code**: Terraform builds everything around the app on the homelab k3s cluster, and builds a complete AWS environment (VPC + EKS) that can be created and destroyed on demand.
- **Operations**: replicated storage, nightly backups with a tested restore, secrets in HashiCorp Vault outside the cluster, metrics, logs, dashboards and alerts delivered to Discord.

2	~90 s	301	57
independent Jenkins pipelines	from "Deploy" commit to running pods	automated tests (235 backend, 66 frontend)	AWS resources, created and destroyed by Terraform

Requirements and how they are met

Requirement	Where
Separate frontend and backend repositories	`tripsync-backend`, `tripsync-frontend`; split from the original combined repo with `git subtree split`, so history was kept. The old repo is archived.
A Jenkinsfile in each app repo	Two jobs, each polling its own repo; the stages are described in chapter 4.
Containerisation	A Dockerfile per app (Gunicorn for Flask; nginx for the React build); `docker-compose.yml` for local development.
Kubernetes deployment	The Helm chart `k8s-helm/tripsync`, deployed to k3s by Argo CD and to EKS by Jenkins.
Infrastructure as code	`terraform/k3s` (homelab add-ons) and `terraform/aws` (own VPC module and EKS).
Secrets management	HashiCorp Vault in its own container outside the cluster (AWS KMS auto-unseal), synced into Kubernetes by the External Secrets Operator; the chart generates its own secrets. Nothing secret in git.

Monitoring; project context file	kube-prometheus-stack, Loki + Alloy, custom alerts and dashboard, Alertmanager → Discord; `CLAUDE.md` in `tripsync-devops`.

2. Architecture

Two flows matter: how a **request** reaches the app, and how a **change** reaches the cluster.

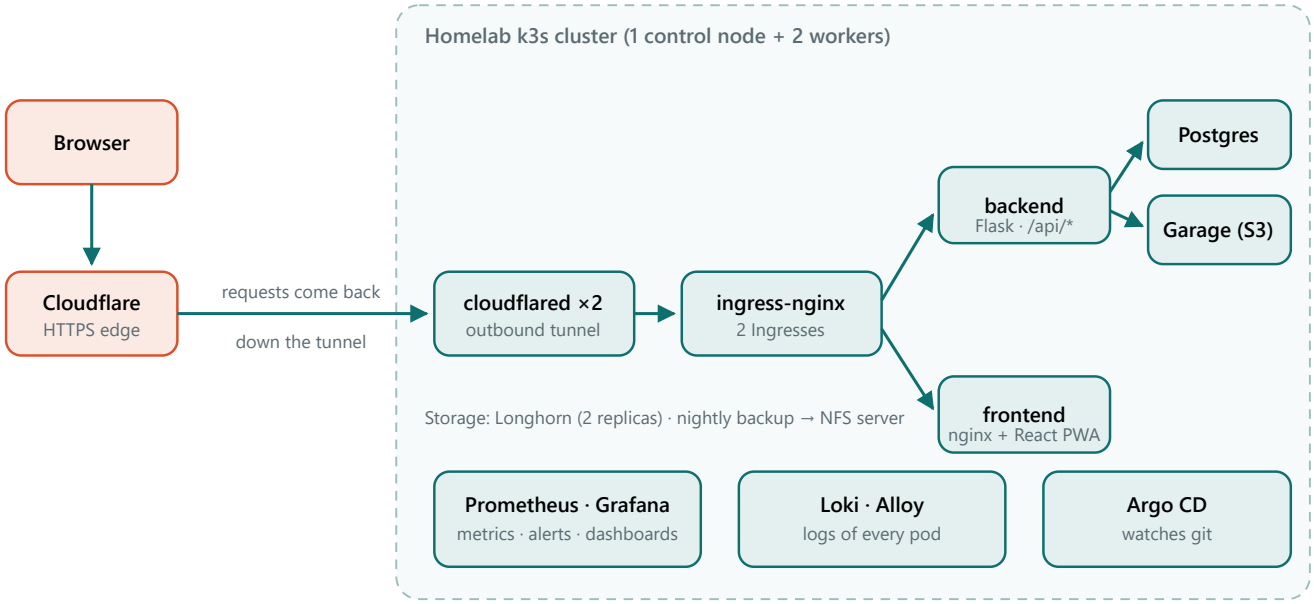


Figure 1. Request path. No router ports are open: cloudflared dials out to Cloudflare and requests come back over that connection.

The homelab

Machine	Role
k3s control node	Kubernetes API; tainted `NoSchedule`, so workloads run on the workers. Traefik is disabled in favour of ingress-nginx.
2 × k3s workers	All app and platform pods; each has a dedicated 100 GB disk for Longhorn (6 GiB RAM each).
Jenkins controller (LXC)	Jenkins UI and job configuration; 0 executors, so nothing builds on it.
Jenkins agent (VM)	Runs every build: Docker, Helm, kubectl, git; label `k3s-agent`.
NFS server	Nightly backup target (`nfs-client` StorageClass).
Vault (LXC, Docker)	HashiCorp Vault 2.0.4 for the external secrets, outside k3s on purpose; reachable over Tailscale only; unseals itself with an AWS KMS key.

Admin access (Jenkins, Grafana, Argo CD, kubectl, Vault) is over **Tailscale** only. The public reaches only the app, through the Cloudflare Tunnel.

The cloud environment

The same chart also runs on **AWS EKS** (eu-central-1), built from scratch by `terraform apply` (57 resources at the last full run, about 20 minutes) and removed by `terraform destroy`. EKS is deliberately ephemeral: it is created for a test session and destroyed afterwards, since its control plane, NAT gateway, node and load balancer all bill by the hour. Each app pipeline has a `TARGET` parameter (`k3s` by default, or `eks`). On EKS photos go to a real **S3 bucket**, with the backend pod getting its permissions through **EKS Pod Identity** (no access key anywhere), and the routing key can come from the same Vault over Tailscale. Both were added in November and are built but not yet run on EKS.

3. The application

Kept brief: the course is about delivery and operations, but the app's design shaped several of the infrastructure decisions.

Backend: tripsync-backend

- Python 3.11, Flask (app factory), Gunicorn (1 worker, 4 threads).
- PostgreSQL 18 via SQLAlchemy; schema changes are Alembic migrations.
- JWT auth with **per-device sessions**: a 7-day token (long on purpose for offline use) that is swapped for a new one on every online start; reusing an old one ends that session (stolen-token detection); “log out on all devices”. scrypt password hashes, rate limits.
- Photos in S3-compatible storage (Garage on k3s, AWS S3 on EKS), re-encoded with Pillow, metadata/GPS stripped, served by signed URLs.
- Prometheus metrics at `/metrics` (cluster-internal only).

Frontend: tripsync-frontend

- React + Vite, built into static files served by nginx, running as a non-root user.
- Installable PWA: a service worker keeps the app available offline.
- Offline-first store: every edit is a small “op”, applied on screen at once, queued in IndexedDB and sent in order when online.
- Map-first UI: Leaflet + MapLibre vector tiles, drag-and-drop itinerary, light/dark/dim themes.

Features

- Trips → days → stops on a map; place search with photos, opening hours and Wikipedia summaries.
- Routes per day: walking, cycling, driving (OpenRouteService) and public transport (Transitous).
- Ideas list, lodging, notes and checklist, 16-day weather warnings, expenses in any currency (ECB rates).
- **Shared trips**: owner / editor / viewer, invite links that expire after 7 days, **expense splitting** with “settle up”.
- Personal ratings and own photos per stop; export as a PDF, as a single offline HTML “keepsake”, or as a Google Maps route.

Design choices with infrastructure consequences

Choice	Consequence
IDs created on the device (UUIDs); creating twice with the same ID is a no-op	Offline edits and retries are safe; the sync queue can resend without creating duplicates.
All outside services go through the backend	One shared cache, API keys stay on the server, call rates are under our control (the routing key allows 200 routes/day).
Same origin everywhere (`/api` on the same host)	No CORS in production; the two Ingresses and the compose gateway reproduce the same routing.
PWA + service worker	Requires HTTPS, one reason for the Cloudflare Tunnel.
Photos in object storage, not the database	The database and its backups stay small; the same S3 API works on AWS.

4. Continuous integration: the Jenkins pipelines

Each app repo has its own Jenkinsfile, and each runs as its own Jenkins job on the agent. The two are deliberately alike, so one explanation covers both.

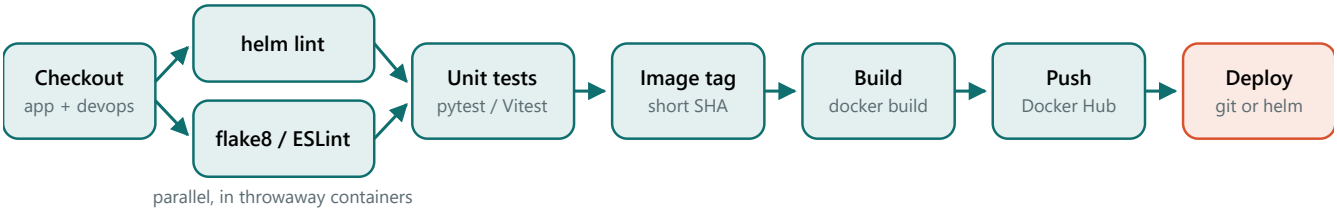


Figure 2. Stages of both pipelines.

Stages

1. **Checkout**: the app repo (polled every 5 minutes) and a plain `git clone` of `tripsync-devops` for the chart. The clone is a shell command on purpose; see chapter 13.
2. **Lint**, in parallel and in throwaway containers: `helm lint` of the chart twice (as EKS and as k3s, with `values-k3s.yaml`), plus flake8 (backend) or ESLint (frontend). Containers run as the Jenkins user (`-u $(id -u):$(id -g)`) so no root-owned files are left in the workspace.
3. **Unit tests**. Backend: a throwaway `postgres:18-alpine` on a per-build Docker network, `flask db upgrade` (every migration runs against real Postgres), then pytest. Frontend: Vitest. Both publish JUnit XML to Jenkins.
4. **Image tag** = the short Git SHA of that app repo, so every build gets a unique, traceable tag; `latest` is never used to deploy (Kubernetes would see no change and not pull).
5. **Build and push** to Docker Hub (`swagarot/tripsync-backend` , `swagarot/tripsync-frontend`); `docker login --password-stdin` , `docker logout` in `post { always }` .
6. **Deploy**, depending on `TARGET` :
   - `k3s` (default, and every polling build): commit the new tag to git, so Argo CD deploys it (chapter 5).
   - `eks` : write a separate kubeconfig with AWS credentials injected only in this branch, then `helm upgrade --install --atomic` .

Both run inside `lock('tripsync-release')`, so the two jobs never deploy at the same moment.

Why polling and not webhooks

Jenkins is only reachable over Tailscale, so GitHub cannot call it. Polling every 5 minutes is the honest trade-off: no inbound exposure, at most a few minutes of delay.

Credentials

All in the Jenkins credential store and referenced by ID: a GitHub token (clone, and push the deploy commit), a Docker Hub access token, and the access key of a dedicated IAM user `jenkins-eks` that can only describe the cluster, plus a namespace-scoped EKS access entry. Secrets are used in single-quoted `sh` blocks so Groovy never interpolates them into logs, and the git token is passed through `gitUsernamePassword` (GIT_ASKPASS), never in a URL.

5. Continuous delivery: GitOps with Argo CD

Since 31 October 2026 the homelab cluster is deployed the GitOps way: git says what should run, and Argo CD, inside the cluster, makes it so.

Push versus pull

Before: push

Jenkins ran `helm upgrade` against the cluster. It needed admin credentials to the cluster, and the cluster was "whatever the last upgrade left". A manual `kubectl edit` stayed until someone noticed.

After: pull

Jenkins only writes the new image tag to git. Argo CD renders the chart from git every minute, compares it with the cluster and applies the difference. Jenkins no longer talks to the k3s API to deploy.

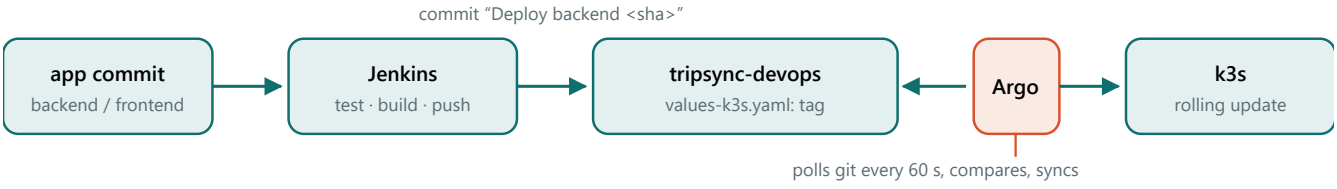


Figure 3. A deploy is a commit. Argo CD pulls it; nothing pushes into the cluster.

How it is set up

- Installed by Terraform (`terraform/k3s/argocd.tf`): argo-cd chart 10.11.6 (Argo CD 3.5.6) without Dex or the notifications controller, and an **Application** `tripsync` via the argocd-apps chart.
- The Application points at `k8s-helm/tripsync` on `main` with `values.yaml` + `values-k3s.yaml`. That second file holds the two image tags and is the only file Jenkins edits. Its git history is the deploy history, and a rollback is `git revert`.

- **Automated sync** with **prune** (things removed from git are deleted) and **self-heal** (manual changes in the cluster are undone). Up to 3 retries with backoff.
- Argo CD reads the devops repo with a **read-only GitHub deploy key** kept only in a Kubernetes Secret created by hand (set up while the repos were private; they are public since 12 November).
- The UI is private: a NodePort reachable over Tailscale, deliberately with no Ingress.
- Jenkins edits the tag with `yq`, commits only if something changed, and on a refused push (the other job pushed first) fetches, redoes the change and retries.

What had to change in the chart

Argo CD renders the Helm chart **without access to the cluster**, so Helm's `lookup` function, which asks the cluster for existing objects, always returns nothing. The chart used it in two places:

Where	Problem under Argo CD	Fix
Generated secrets (DB password, JWT key, Garage keys)	New random values on every render: the backend would lose its database login and every user would be logged out.	The Application ignores the Secret's `/data` (`ignoreDifferences` + `RespectIgnoreDifferences`). Verified by hash against the running pods.
Backup CronJob	Rendered only if the `nfs-client` StorageClass existed, which `lookup` now always reported missing.	An explicit value `backup.enabled`: off by default (EKS), on in `values-k3s.yaml`.

The Secret and the backup volume are also marked `Prune=false,Delete=false`, so Argo CD never deletes the data or the keys that unlock it.

Taking over a live app with no downtime

A backup was taken first. The rendered chart was then compared with the live release (identical apart from two annotations). The Application was created with auto-sync **off**, and its diff showed nothing to change or prune. Only then was auto-sync switched on. No pod restarted. End-to-end proof: a commit produced “Deploy frontend …” and “Deploy backend …” commits, and Argo CD rolled each out about 90 seconds later.

Why these decisions

Why does Jenkins commit the tag instead of using Argo CD Image Updater?
One less component, and the commit records who deployed what, when.

Why is EKS still push-based?
It is a throwaway cluster that lives for one test session. Installing Argo CD there each time adds nothing, so Jenkins keeps deploying it with `helm upgrade --atomic`.

How is it monitored?
Argo CD's metrics go to Prometheus, with alerts for “out of sync for 15 min”, “unhealthy for 10 min”, “sync failed” and “metrics missing”. They arrive in Discord like every other alert.

6. Kubernetes and the Helm chart

One chart, `k8s-helm/tripsync`, describes the whole app on both clusters. Cluster differences live in values (storage class, image tags, backup on/off), never in separate copies of the chart.

Template	What it creates and why
backend-deployment	Backend Deployment. A `migrate` init container runs `flask db upgrade` before the app starts, and a `storage-init` init container bootstraps Garage (layout, bucket, key; every step repeatable). Runs as non-root.
frontend-deployment	nginx serving the built PWA, as non-root (the `nginx-unprivileged` image, UID 101, `runAsNonRoot`). It keeps port 80 through the pod sysctl `net.ipv4.ip_unprivileged_port_start=0`, which applies only inside the pod's own network namespace.
postgres	A **StatefulSet** with a headless Service and its own volume (2Gi).
garage	Photo object storage (S3 API), a 1-node StatefulSet with a 10Gi volume.
secret	Postgres password, JWT key and Garage keys, generated once and never in git.
ingress	**Two** Ingresses: `/api(/|$)(.*)` with a rewrite to the backend, and plain `/` to the frontend.
backup	Nightly CronJob plus its NFS volume (chapter 8). Enabled per cluster by `backup.enabled`.
external-secrets	ExternalSecrets for the routing key and the tunnel token, read from Vault. Rendered only if the External Secrets Operator is installed (`.Capabilities`), with `creationPolicy: Orphan`, so the Secret survives if they are removed.
backend-serviceaccount	On EKS only (`s3.enabled`): the identity EKS Pod Identity gives the S3 role to.
servicemonitor, prometheus-rules, grafana-dashboard	Scrape config, alert rules and a dashboard shipped with the app. They render only if the cluster has the Prometheus Operator, so EKS still installs.

Decisions worth defending

Why a StatefulSet for Postgres?

It gives a stable identity and a volume that outlives the release. Above all, it never runs two copies at once during an update; a Deployment's rolling update would briefly run two Postgres processes on the same files.

Why two Ingresses?

ingress-nginx annotations apply to a whole Ingress object. With one combined Ingress, the `/api` rewrite also rewrote requests for JavaScript files, so nginx answered them with `index.html`: a white screen and a MIME error.

Liveness versus readiness?

Liveness (`/health/live`) only asks whether the process is stuck; it must not check the database, or a database outage would restart every pod in a loop. Readiness (`/health/ready`) runs `SELECT 1` and only takes the pod out of the Service. Garage's readiness is a TCP check, because its `/health` fails until it has been set up, and setup goes through its Service: a deadlock otherwise.

Why migrations in an init container?
The new schema is in place before new code starts. If a migration fails, the new pod never becomes ready and the old one keeps serving. During a rolling update old and new code run side by side, so migrations only add, with new columns nullable or defaulted (expand/contract). An advisory lock stops two pods from migrating at the same time.

Why generate secrets in the chart?
Nobody types them and they never appear in git. They are generated once and then kept: Postgres only reads its password on first start, and a new JWT key would log everyone out. The Secret has `helm.sh/resource-policy: keep`, so uninstalling never separates it from the data it unlocks. Secrets from outside (tunnel token, routing API key, Discord webhook) live in Vault and are synced in by External Secrets; without Vault they are created by hand with kubectl. The Argo CD deploy key is the one still made by hand.

Deployment, ReplicaSet, Pod

A Deployment is the desired state ("1 copy of backend:10cd680"). For each version of its pod template it creates a ReplicaSet, which keeps N identical pods running. A new image tag means a new ReplicaSet: the Deployment scales it up and the old one down to zero, and readiness probes keep traffic away from pods that aren't ready yet. We keep the last 3 ReplicaSets (`revisionHistoryLimit: 3`). With GitOps the real rollback is `git revert`, since self-heal would undo a manual `kubectl rollout undo`.

7. Infrastructure as code: Terraform

Two root modules, each with remote state in S3, locked with S3's native lock file (`use_lockfile`, Terraform ≥ 1.10), which replaced the deprecated DynamoDB table. The lock was proven with two concurrent plans: the second got *412 Precondition Failed*.

terraform/k3s: the homelab platform

File	Manages
main.tf	The `tripsync` namespace. Terraform owns it, so the chart never creates it.
longhorn.tf	Longhorn 1.12.1, replicated block storage (2 replicas).
monitoring.tf	kube-prometheus-stack (Prometheus, Alertmanager, Grafana, node exporter, kube-state-metrics) and the Longhorn alerts.
logging.tf	Loki (single binary, 7-day retention) and Grafana Alloy as a DaemonSet.
tunnel.tf	cloudflared as a **DaemonSet**: one connector on each worker (the control node is tainted, so 2 today), read-only root filesystem. It was a 2-replica Deployment until both pods once landed on the same worker; a DaemonSet can't do that.
argocd.tf	Argo CD and the `tripsync` Application.

terraform/vault and terraform/vault-config: secrets

`terraform/vault` creates the AWS KMS key that unseals Vault (kept permanently, deletion-protected, yearly rotation), an IAM user that may only use that key, and installs the External Secrets Operator in k3s. `terraform/vault-config` configures Vault itself: a KV store, read-only policies, the Kubernetes login (Vault

checks a pod's ServiceAccount token with the k3s API), the `ClusterSecretStore`, Prometheus scraping and alerts for Vault, and its nightly backup. Vault runs from `vault/compose` on a separate homelab machine; `vault/README.md` is its runbook.

Helm charts are installed by `helm_release` with values kept as plain YAML files (`monitoring/`, `argocd/`), so they can be read and diffed like any other config. Chart versions are pinned.

terraform/aws: VPC + EKS

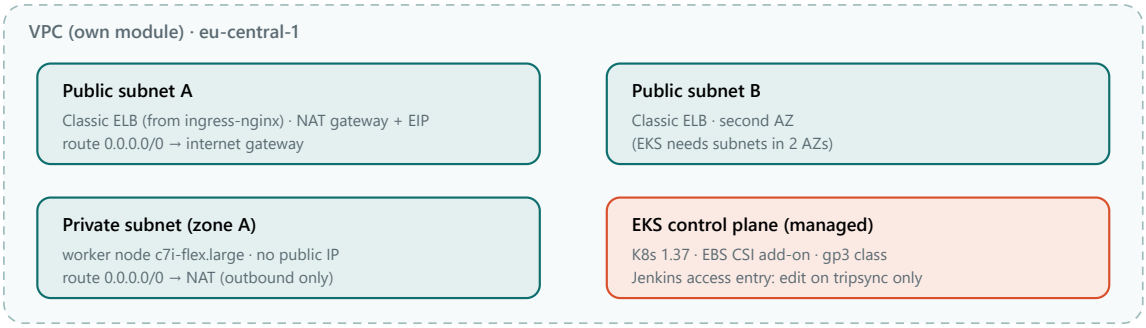


Figure 4. The AWS environment: 2 public subnets and 1 private subnet, cheaper than the usual 2 + 2.

- **Own VPC module**, written step by step instead of the community module: VPC with DNS support and hostnames, internet gateway, 2 public subnets, 1 private subnet, NAT gateway with an Elastic IP, route tables and associations, and subnet tags for load balancers. NAT can be switched off with `count`.
- **EKS** through the official module: nodes in the private subnet; ingress-nginx, the namespace, the EBS CSI driver and an encrypted gp3 StorageClass all in Terraform, so a fresh cluster is ready for the first pipeline run with no manual steps. The account is on the Free Plan, so the node is a free-tier-eligible `c7i-flex.large` (`t3.medium` is refused).
- **Photos in S3** (`photos.tf`, added in November): a private bucket, an IAM role allowed only to read and write its objects, and an EKS Pod Identity association giving that role to the backend's ServiceAccount.
- **Least privilege for Jenkins**: an IAM user that can only describe the cluster, plus an EKS access entry limited to edit rights in the `tripsync` namespace. The user and its key are made by hand, so the secret never lands in Terraform state.

Proven, clean slate both times

10 Oct: apply (54 resources), both pipelines with `TARGET=eks`, app served through the ELB, destroy with no leftovers. **28 Oct** (after Postgres/Garage were added): apply (57 resources), chart installed in 38 s, PVCs bound on encrypted EBS, all migrations on an empty database, a full smoke test through the ELB. Teardown removed the volumes *before* destroy so no billed disk was orphaned. AWS was verified empty afterwards: 0 volumes, load balancers, EIPs and instances.

8. Storage, data and backups

Longhorn: replicated volumes

Longhorn turns the workers' disks into Kubernetes volumes. Each volume has a full copy on both workers, and every write goes to both before it is confirmed. If a worker dies, the pod restarts on the other worker with the surviving copy (tested: Prometheus and Alertmanager were back in under 50 seconds with their data intact).

Prometheus, Alertmanager, Loki, Postgres and Garage all use it.

Why not NFS for everything?
Prometheus documents NFS as unsupported (corruption risk), and databases want a block device with one writer. NFS is used for what it is good at: being the backup target.

> **A replica is not a backup**
> Replicas protect against a dead disk or node. A mistake (a bad DELETE, a broken migration, a Longhorn bug) hits both copies instantly. Only a backup undoes that.

Nightly backup

- A CronJob at 03:30 (Asia/Jerusalem) with `concurrencyPolicy: Forbid`.
- An init container runs `pg_dump -Fc` (kept 14 days), then `rclone sync` mirrors the photo bucket, keeping deleted photos for 30 days in a dated folder.
- Target: an NFS volume (30Gi) marked to survive uninstall. A `LAST_RUN.txt` marker and the alert *TripsyncBackupMissing* (>26 h since the last success).
- **Vault** is backed up as an encrypted Raft snapshot twice a night: next to Vault, and by a CronJob in k3s onto the NFS server (an unprivileged container can't mount NFS, so the cluster pulls the snapshot over Tailscale). Restoring needs the KMS key, so the snapshots are safe on NFS. Alert: *VaultBackupMissing*.
- **Restore tested, not assumed**: the dump was restored into a scratch database with identical row counts and migration version. The restore guide is `docs/backup-restore.md`.

Known weak spots, chosen with open eyes

- Shared fate: every stateful service sits on Longhorn. Mitigated by alerts on degraded or faulted volumes, node health and disk usage, and by the backup living outside Longhorn.
- Only 2 storage nodes: while one is down, every volume has one copy. That is exactly what *LonghornVolumeDegraded* reports.
- The NFS server is in the same building. Offsite copies (Cloudflare R2) are the next step.
- Garage runs as one node: tests showed 2 nodes refuse writes when one is down; real write availability needs 3. Postgres is the availability limit anyway.

9. Networking and exposure

Public: Cloudflare Tunnel

cloudflared opens an **outbound** connection to Cloudflare; visitors connect to Cloudflare over HTTPS and requests come back down that connection to ingress-nginx. No router ports are open, no public IP is needed, and HTTPS (required for the service worker and app install) comes for free. cloudflared runs as a DaemonSet, one connector on each worker, so losing a worker never takes the tunnel down.

Because every request now arrives from a cloudflared pod, the backend's rate limits key on `CF-Connecting-IP`, which Cloudflare sets and a client cannot forge through the tunnel.

Private: Tailscale + NodePorts

Admin tools have no Ingress at all. Grafana (port 30300) and Argo CD (port 30443) are NodePorts, reachable on the nodes' LAN and Tailscale addresses but **not** through the tunnel, which only forwards to ingress-nginx. Jenkins and kubectl are reachable over Tailscale only.

Found and closed: an accidental exposure

Grafana used to be served at `/grafana` through a host-less Ingress. A host-less Ingress matches *every* hostname, including the public tunnel hostname, so Grafana's login page was on the internet. The same mechanism had earlier exposed the backend's `/api/metrics`. Both are closed: the API Ingress regex excludes `metrics`, and Grafana moved to a private NodePort. Lesson: with a tunnel in front, every Ingress is public unless proven otherwise.

Caching gotcha

nginx sends `no-cache` for `index.html`, `sw.js` and the manifest, and long-lived immutable caching for hashed assets. Cloudflare's default browser cache TTL (4 h) overrode `no-cache` on `sw.js`, so app updates were delayed. It is now set to “Respect Existing Headers”.

10. Observability: metrics, logs and alerts

Monitoring runs on the homelab only. EKS is ephemeral and its 4 GiB node is too small; Amazon Managed Prometheus is noted as an option.

Metrics

- kube-prometheus-stack: Prometheus (7 days / 6 GB, Longhorn volume), Alertmanager, Grafana, about 22 scrape targets.
- Backend metrics labelled by **route template** (`/trips/<uuid:trip_id>`), not by URL; otherwise every trip ID would be a new time series forever.
- Counters for outside-API calls versus cache hits.
- Grafana has no volume: dashboards and data sources are code (ConfigMaps), so it can be recreated at any time.

Logs

- Grafana Alloy on every node tails the pod logs, labels them (namespace, pod, container, app, node) and ships them to Loki.
- Loki single binary, 7-day retention, on Longhorn.
- Alloy instead of Promtail, which is deprecated.

Alerts (on top of the stack's ~130 built-in rules)

Alert	Fires when
TripsyncBackendDown / FrontendDown	No backend target, or 0 available frontend pods, for 2 min (critical)
TripsyncHighErrorRate / SlowResponses	>5% 5xx for 5 min; p95 > 1 s for 10 min

TripsyncDatabaseDown / PhotoStorageDown	Postgres or Garage not available
TripsyncBackupMissing	No successful backup for 26 h
Longhorn: VolumeDegraded / Faulted / NodeNotReady / DiskFilling / MetricsMissing	A volume down to one copy or none; a storage node down; disk > 70% (Longhorn stops at 75%); no Longhorn metrics
ArgoCD: AppOutOfSync / AppUnhealthy / SyncFailed / MetricsMissing	Cluster differs from git for 15 min; app unhealthy for 10 min; a failed sync
VaultSealed / VaultDown / VaultBackupMissing	Vault restarted and could not auto-unseal; Prometheus can't reach it over Tailscale; no snapshot reached NFS for 26 h
AlloyRunawayReads	The log collector reads more than 50 MB/s from disk for 5 min (critical; see incident 5)

Every alert except the always-on Watchdog goes to **Discord**, grouped by namespace and alert, with "resolved" messages. The webhook URL lives in Vault and is synced into a Kubernetes Secret.

Is it tested?
Yes. The backend was scaled to 0: the alert was active in Alertmanager 3 minutes later (the target disappearing plus the 2-minute `for:`) and arrived in Discord. Scaled back up, it cleared, and the resolved message followed at the next group interval.

Why is every "…MetricsMissing" alert there?
An alert on a metric that no longer exists stays silent forever. `absent()` turns silence into a signal.

11. Security

Area	Measures
Secrets	None in git. The chart generates its own and keeps them; external ones live in HashiCorp Vault, outside the cluster (a compromised cluster doesn't take the secret store with it), auto-unsealed by AWS KMS and synced in by External Secrets with a 1-hour, read-only Vault token. Jenkins credentials referenced by ID; the git token goes through GIT_ASKPASS; Argo CD has a read-only deploy key.
Exposure	Only the app is public, through the outbound tunnel. Admin UIs are on Tailscale only, and `/metrics` is excluded from the public Ingress.
Least privilege	Separate IAM user for Jenkins (describe cluster only) plus a namespace-scoped EKS access entry; an IAM user that can only use the Vault KMS key; on EKS the backend gets its S3 role through Pod Identity, with no stored key; Vault backup tokens can only take snapshots; Argo CD only reads git.
Containers	Both app images run as non-root (backend UID 10001, frontend nginx UID 101) and the chart enforces it (`runAsNonRoot`); multi-stage frontend build; cloudflared with a read-only root filesystem; pinned image versions.

Authentication	Per-device sessions with token rotation and reuse detection, logout on one or all devices; scrypt hashes; the same response time for an unknown email and a wrong password; rate limits on login, sign-up, search and uploads.
Authorisation	One module answers “can this user see this trip, with which role”. Not on the trip = 404 (IDs don't leak); a viewer trying to edit = 403. A test calls every endpoint as an outsider and as a viewer.
Uploads	Images must decode, are re-encoded with metadata and GPS stripped, size limits apply, and they are served by signed, expiring URLs.
SSRF	The one endpoint that fetches a URL is locked to Wikimedia: allow-listed prefix, redirects checked by hand, image types only, 5 MB cap. Tests include the cloud metadata address.

12. Testing

235	66	2	1
backend tests (pytest)	frontend tests (Vitest)	EKS clean-slate runs	real restore, row counts matched

- **Real database in CI.** Backend tests run against Postgres 18 after all migrations, the same schema as production, not SQLite.
- **Isolation tests.** A second user calls every ID-taking endpoint on the first user's data and must get 404; viewers must get 403 on every write.
- **Query-count test.** Guards against N+1 queries: a trip page made 40 SQL queries for 5 days × 6 stops; it is now constant (≤ 11) whatever the trip's size.
- **Frontend logic.** The offline queue against a fake server, conflict handling, route caching, weather rules, expense splitting to the cent.
- **End to end** (Playwright on the local compose stack): three users, invite while logged out, join, split expenses, a viewer on a phone, removal, export offline. Offline reload and sync were checked against the public URL.
- **Deploy-it-yourself test.** The quick path in `docs/deploy-yourself.md` was run as written on a fresh k3s with no keys: register, trip, stop, photo upload all worked; routes fell back to straight lines, as designed.
- **Infrastructure tests.** A Longhorn failover (cordon a worker, delete pods), an alert fired end to end, a backup restored into a scratch database, two Terraform plans at once to prove the state lock.

13. Incidents and lessons learned

The most useful part of running something for real: things went wrong, monitoring or process caught them, and each left a lesson.

1. Eleven days of "failed" backups

From 17 to 28 October every nightly backup was marked failed. The cleanup step (`rclone delete --rmdirs`) exited with an error whenever a folder still held recent photos. The dumps and the photo mirror had been written every night; only the last step failed. Monitoring worked: *TripsyncBackupMissing* and *KubeJobFailed* sent 60 Discord messages, all delivered. Nobody read them. The fix was reproduced with the same rclone image (old command exits 1, new exits 0) and verified with a manual run.

Lessons: test the cleanup path, not only the happy path; an alert needs an owner who reads it.

2. The backup CronJob deleted during the Argo CD switch

The chart change (backup now off by default) was pushed before the Jenkinsfile change. Both Jenkins jobs then built and ran the *old* pipeline (`helm upgrade`) with the *new* chart, and Helm deleted the CronJob. The data survived because the volume is marked `keep` , and Argo CD's first sync recreated the CronJob.

Lesson: push the pipeline change before, or together with, the chart change it depends on.

3. Jenkins building on the wrong repo

Both jobs started on every commit to `tripsync-devops` , although that checkout had `poll: false` and the polling logs showed only each job's own repo. The proof came from Docker Hub: images were re-pushed about 5 minutes (one polling interval) after each devops commit. With GitOps every deploy commit then rebuilt both apps. Fix: clone the devops repo with a plain shell `git clone` , since Jenkins only polls repos checked out by its git steps. Verified: no rebuilds afterwards.

4. Grafana on the public internet

See chapter 9: a host-less Ingress matched the public hostname. It was found during the Argo CD work and closed by moving the admin UIs to private NodePorts.

5. The log collector that took down storage

On 6 November the k3s API timed out with SQLite "database is locked" errors and every Longhorn volume degraded. The cause was Grafana Alloy: close to its 256 MiB memory limit, the kernel kept evicting its pages and reading them back, about 900 MB/s from disk (549 GB read for 3 GB of logs), which saturated the Proxmox storage all machines share. A restart had only reset the growth the first time. Fix: Alloy upgraded, memory limit raised to 512 MiB with `GOMEMLIMIT` , and a new alert, *AlloyRunawayReads*.

Lesson: a memory limit below what a process needs resident shows up as disk I/O, not as an out-of-memory kill; and on shared storage one noisy pod hurts everything.

6. A storage engine missing on one worker

After the cluster was shut down to add GPUs (2 November), Longhorn's engine binary was missing on one worker, so its volume replicas kept failing and rebuilding. *LonghornVolumeDegraded* fired for 4 days before anyone looked. Deleting that node's engine-image pod made Longhorn redeploy it; all volumes were healthy again within minutes.

Lesson: again, an alert needs an owner; and after maintenance, drain and return one node at a time.

7. Smaller ones, each a line in the runbook

- A combined Ingress with a rewrite served HTML for JS files (white screen), hence the two Ingresses.
- Grafana's 384 Mi memory limit was too low (it sat at ~372 Mi, health timed out, 503s), so it was raised to 768 Mi.
- Longhorn mounts failed while `multipathd` grabbed the iSCSI devices; it is disabled on the workers.

- The tunnel token Secret held literal quotes (Windows `cmd` keeps `'`): “token is not valid”.
- A deploy ran the new chart with an old backend image; the migrate init container failed and `--atomic` rolled back. Hence the deploy order: chart, then backend, then frontend.
- Garage's HTTP health check deadlocked its own setup, so readiness is a TCP check.

14. Project timeline

When	Milestone
Weeks 1–2	Flask + React app, Dockerfiles, docker-compose; homelab k3s with ingress-nginx; first Helm chart (deployments, services, ingress, probes, limits); Terraform for the namespace; Jenkins controller and agent; one combined pipeline with SHA tags.
Week 3	Split into backend and frontend repos (history kept); a Jenkinsfile and job per repo, triggered by SCM polling; shared deploy lock; old job and repo retired.
Week 4 (10 Oct)	AWS with Terraform: own VPC module, EKS, ingress-nginx; both pipelines deploy to EKS through `TARGET`; clean-slate apply → deploy → destroy.
Week 5 (12–14 Oct)	Longhorn; Prometheus, Alertmanager, Grafana; Loki + Alloy; custom alerts and dashboard; Discord; failover and alert tests.
Weeks 6–7 (16–17 Oct)	Multi-account backend on Postgres with migrations; CI tests on real Postgres; offline-first PWA; Cloudflare Tunnel, live in public; Garage photo storage; nightly backups with a tested restore; maps, routes, transit, weather, expenses.
Sprint (28 Oct)	Shared trips and expense splitting; PDF / keepsake / Google Maps export; backup incident fixed; Terraform S3-native state locking; second full EKS run.
31 Oct	GitOps with Argo CD; admin UIs made private; Jenkins polling fix; ReplicaSet history trimmed.
1 Nov	Documentation and presentation drafts.
6 Nov	Per-device sessions with token rotation; S3 + Pod Identity for EKS; HashiCorp Vault outside the cluster with KMS auto-unseal and External Secrets; storage incident fixed (Alloy).
8 Nov	Vault live as the source of the external secrets, with nightly snapshots to NFS; frontend moved to non-root nginx; the deploy-it-yourself guide tested on a fresh k3s.
12 Nov	READMEs and documentation updated; repositories made public.

15. Limitations and future work

Limitation	Next step
Backups are in the same building	Offsite copy to Cloudflare R2 (10 GB free).
Postgres has one instance; failover takes about a minute and depends on Longhorn	Replication independent of Longhorn, e.g. CloudNativePG with 2 instances on local disks.
Only 2 storage nodes	A third node would allow 3 replicas and a 3-node Garage with real write availability.
ingress-nginx is retired (last release March 2026)	Move to the Gateway API with a maintained controller.
EKS has no monitoring and is push-deployed	Amazon Managed Prometheus for app metrics; Argo CD there only if EKS became long-lived.
Images on Docker Hub; EKS pulls through NAT	ECR for the AWS path.
S3 + Pod Identity and EKS reading from Vault are built but not yet run on EKS	One more EKS session to prove them end to end.
Vault is a single instance; its root token is still kept	Revoke the root token (it can be regenerated from the recovery keys); tighten the Tailscale ACL to exactly the ports needed.
In-memory rate limits are per pod	A shared store (Redis) if the backend scales out.

Appendix A. Versions and repository layout

Component	Version
Argo CD (chart)	3.5.6 (10.11.6)
kube-prometheus-stack	92.3.0
Loki (chart) / Alloy chart	3.6 (7.3.0) / 1.15.0
HashiCorp Vault	2.0.4
External Secrets Operator	2.14.0
Longhorn	1.12.1
ingress-nginx chart	4.15.1
cloudflared	2026.10.1

```
tripsync-devops/
├── k8s-helm/tripsync/   Helm chart
│   ├── values.yaml       defaults (EKS)
│   ├── values-k3s.yaml   homelab: image tags
│   │                     (edited by Jenkins)
│   ├── templates/        Kubernetes objects
│   └── dashboards/       Grafana JSON
├── argocd/              Argo CD, the app
├── monitoring/          stack, Loki, Alloy,
│                        Longhorn alerts
├── terraform/
│   ├── k3s/             homelab platform
│   ├── aws/             VPC module + EKS
│   ├── vault/           KMS key, ESO
│   └── vault-config/    Vault policies,
│                        logins, backups
├── vault/               Vault compose
│                        + runbook
├── compose/, docker-compose.yml
├── docs/                deploy yourself,
```

PostgreSQL	18.6
Garage	v2.4.1
EKS Kubernetes	1.37
Node (frontend build)	24
Python (backend)	3.11

```
│                          backup & restore
├── CLAUDE.md              project context
└── walkthrough.md         learning log

tripsync-backend/    Flask API, tests,
                     Dockerfile, Jenkinsfile
tripsync-frontend/   React PWA, tests, nginx,
                     Dockerfile, Jenkinsfile
```